

* Always Remember: → Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. - vHixem

* Problem: -

Given an integer array **height[]** where **height[i]** represents the height of the **i-th** stair, a frog starts from the **first stair** and wants to reach the **last stair**. From any stair **i**, the frog has two options: it can either jump to the **(i+1)th** stair or the **(i+2)th** stair. The cost of a jump is the absolute difference in height between the two stairs. Determine the **minimum total cost** required for the frog to reach the last stair.

(stair) idx $\rightarrow$ 0 1 2 3

Input: heights[] = [20, 30, 40, 20], $n = 4$

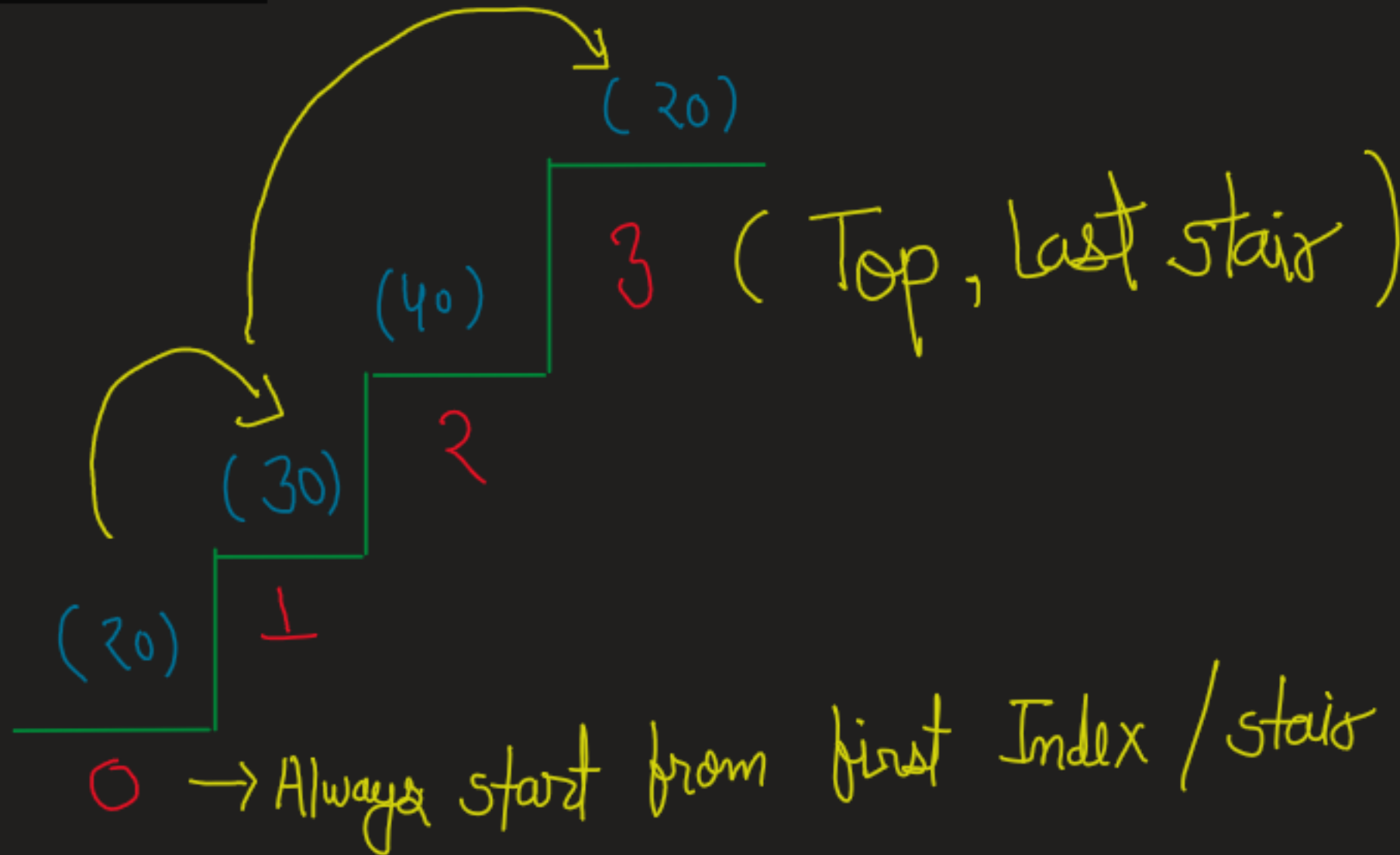
Output: 20

Explanation: Minimum cost is incurred when the frog jumps from stair 0 to 1 then 1 to 3:

jump from stair 0 to 1: cost = $|30 - 20| = 10$

jump from stair 1 to 3: cost = $|20 - 30| = 10$

Total Cost = $10 + 10 = 20$



$$\begin{aligned} \rightarrow \text{Total cost} &= \text{abs}(20-30) + \text{abs}(30-20) \\ &= 10 + 10 \Rightarrow \underline{20} \end{aligned}$$

$\rightarrow$ If array size is n then top is $n-1$

(index) stairs $\rightarrow$ 0 1 2 3
(values) Heights $\rightarrow$

20	30	40	20
----	----	----	----

, $n=4$

Jump1Step = $\text{abs}(\text{Height}[1] - \text{Height}[2])$ + get result from 2th idx $\rightarrow$ Find Recursively
Jump2Step = $\text{abs}(\text{Height}[1] - \text{Height}[3])$ + get result from 3th idx $\rightarrow$

$\rightarrow$ In general:-

Jump1Step = $\text{abs}(\text{Height}[i] - \text{Height}[i+1])$ + get result from $i+1$ idx $\rightarrow$ Find Recursively
Jump2Step = $\text{abs}(\text{Height}[i] - \text{Height}[i+2])$ + get result from $i+2$ idx $\rightarrow$

We need minimum cost so take $\Rightarrow \min(\text{Jump1Step}, \text{Jump2Step})$

• So we got small idea, what should do.

* Recursive Code:-

```
int Rec(i, height) {  
    if (i == n-1) return 0  
    if (i > n) return INT_MAX  
  
    int jump1step = abs(heights[i] - height[i+1])  
        + Rec(i+1, height);
```

```
    int jump2step = Rec(i+2, height)
```

```
    if (jump2step != INT_MAX) {
```

```
        jump2step += abs(heights[i] - height[i+2])
```

```
    }  
    return min(jump1step, jump2step)  
}
```

